Theory for Advanced MRI

Prof Chris Rodgers*

4th Mar 2026

1 Resources

This lecture is a very brief introduction to the topic of simulations for MRI. For those who want to read more about this, I recommend the following resources:

- Magnetic resonance imaging: Physical principles and sequence design by E. Mark Haacke.[1]. One of the classic textbooks on MRI physics.
- Spin Dynamics by Malcolm Levitt.[2] Theory of spin dynamics including the Bloch equations, and their application in key pulse sequences used in MRI and NMR.
- Principles of Magnetic Resonance by C.P. Slichter.[3] Textbook on magnetic resonance theory with a very clear and consistent mathematical treatment.
- `Pulseq` open-source MRI software. I have borrowed from their excellent tutorials at https://github.com/pulseq/.
  - Hands-on exercises are based on running Python code in Notebooks within the Google Colab environment, therefore, no additional software is required.
  - A Google account and a laptop or tablet connected to the Internet are required.

Note that equivalent theory is used for applications in high-resolution Nuclear Magnetic Resonance (NMR) spectroscopy, laser optics and quantum computing.

2 Notation conventions

- Superscript L denotes quantities in the laboratory frame.
- No superscript denotes quantities in the rotating frame.
- ω means an angular frequency, in units of radians per second.
- ν means a frequency, in units of hertz (cycles per second). Note that $\omega = 2\pi\nu$. Note also that MRI physicists are very sloppy about remembering the word "angular" when describing an angular frequency.

*Email: ctr28@cam.ac.uk

3 Bloch equations in the rotating frame

Recall that the Bloch equations in the laboratory frame are given by:

$$\frac{d}{dt}\begin{pmatrix} M_x^L \\ M_y^L \\ M_z^L \end{pmatrix} = \begin{pmatrix} -\frac{1}{T_2} & \gamma B_z & -\gamma B_y \\ -\gamma B_z & -\frac{1}{T_2} & \gamma B_x \\ \gamma B_y & -\gamma B_x & -\frac{1}{T_1} \end{pmatrix} \begin{pmatrix} M_x^L \\ M_y^L \\ M_z^L \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{1}$$

Consider the situation when we bring an object (or person) into the MRI scanner. The scanner's main magnetic field $B_0 = 1.5 - 11.7\,\mathrm{T}$ is taken to define the z-axis so $B_x = B_y = 0$ and $B_z = B_0$. Ignoring T2 relaxation for the moment ($T_2 \gg 1$) we see that

$$\frac{d}{dt}\begin{pmatrix} M_x^L \\ M_y^L \\ M_z^L \end{pmatrix} = \begin{pmatrix} 0 & \gamma B_0 & 0 \\ -\gamma B_0 & 0 & 0 \\ 0 & 0 & -\frac{1}{T_1} \end{pmatrix} \begin{pmatrix} M_x^L \\ M_y^L \\ M_z^L \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{2}$$

In other words, the magnetization vector $\mathbf{M}^L$ precesses around this axis at the Larmor angular frequency $\omega_0 = \gamma B_0$. This means that in the laboratory frame, the magnetization vector is rotating rapidly around the z-axis. The magnetisation is said to undergo **free precession** under the influence of the main magnetic field B_0.

Since the magnetization is rotating rapidly, it can be difficult to analyze its behavior in the laboratory frame. To simplify the analysis, we switch to a **rotating reference frame** that rotates at the same frequency as the magnetization.

Mathematically we define $\mathbf{M}$ in a reference frame rotating at a chosen *reference frequency* ω_{ref}.

$$M_x(t) = \quad M_x^L(t)\cos(\omega_{\mathrm{ref}}t) + M_y^L(t)\sin(\omega_{\mathrm{ref}}t) \tag{3}$$

$$M_y(t) = -M_x^L(t)\sin(\omega_{\mathrm{ref}}t) + M_y^L(t)\cos(\omega_{\mathrm{ref}}t) \tag{4}$$

$$M_z(t) = \quad M_z^L(t) \tag{5}$$

Define the *resonance offset* $\Delta B_0 = B_0 - \omega_{\mathrm{ref}}/\gamma$ and the *frequency offset* $\Delta\omega = \omega_0 - \omega_{\mathrm{ref}}$, then during a period of free precession (no RF) the rotating frame Bloch equations are:

$$\frac{d}{dt}\begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} = \begin{pmatrix} -\frac{1}{T_2} & \gamma\Delta B_0 & 0 \\ -\gamma\Delta B_0 & -\frac{1}{T_2} & 0 \\ 0 & 0 & -\frac{1}{T_1} \end{pmatrix} \begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{6}$$

This corresponds to precession around the z-axis at the frequency offset $\Delta\omega$, plus longitudinal T_1 relaxation of M_z and transverse T_2 relaxation of M_x and M_y.

If we choose the reference frequency to be exactly on resonance, i.e. $\omega_{\mathrm{ref}} = \omega_0$, then $\Delta\omega = 0$ and the magnetization vector does not precess at all in the rotating frame. This is a very convenient choice for analyzing the effects of RF pulses, which are typically applied at or near the Larmor frequency.

We will use the rotating frame from this point on. The main advantage is that the magnetization vector $\mathbf{M}$ is now precessing at a much slower rate (the frequency offset $\Delta\omega$) which makes it easier to analyze its behavior under the influence of RF pulses and other perturbations.

It also makes it easier to solve the Bloch equations numerically, since we can use larger time steps without losing accuracy.

3.1 RF phasors

Only transverse RF fields oscillating at frequencies close to the Larmor frequency will have a significant effect on the magnetization. These RF fields are often represented as *phasors*.[1]

A phasor is a complex number that represents a sinusoidal function at a pre-defined frequency and having a certain amplitude and initial phase. For example, the sinusoid

$$x(t) = A\cos(\omega t + \phi) \tag{7}$$

is represented as the phasor

$$\tilde{X} = Ae^{i\phi}, \tag{8}$$

where A is the amplitude and ϕ is the initial phase. The time dependence is implicit in the phasor representation, since it is understood that the phasor oscillates at the frequency ω. In MRI the phasor frequency is typically ω_{ref}.

We will often write the phasors as being time-dependent, i.e. $\tilde{X}(t)$. This is a common convention in MRI and NMR, and it allows us to represent microsecond time scale changes in the amplitude and phase of the *pulse envelope*. The carrier frequency $\omega_{\text{ref}} \sim 100 - 500\,\text{MHz}$ with nanosecond time scale oscillations remains implicit in the phasor representation.

3.2 Complex transverse magnetization

We often represent the rotating frame transverse magnetization as a single complex number $M_{xy} = M_x + iM_y$. This allows us to easily represent both its magnitude and phase.

3.3 RF field (B_{RF}) in the rotating frame

When we apply a radiofrequency field, its z-component has negligible impact on the magnetization vector because $|B_{\text{RF}}| \ll |B_0|$. So we only need to consider its transverse components $B_{\text{RFx}}(t)$ and $B_{\text{RFy}}(t)$.

Furthermore, only the component of the transverse field that rotates in the same direction as the precessing magnetization has a significant net effect on the magnetization. This is

$$B_1^+(t) = \frac{1}{2}\left(B_{\text{RFx}}(t) + iB_{\text{RFy}}(t)\right). \tag{9}$$

The other component rotates in the opposite direction so its effects average close to zero over time. This is known as the **rotating wave approximation**.[4] Note that B_1^+, B_{RFx}, and B_{RFy} are all complex phasors. Be careful when looking at the literature because in older papers predating ultra-high field parallel transmit MRI these distinctions were not always made and the notation can be very inconsistent.

[1] https://en.wikipedia.org/wiki/Phasor

3.4 Bloch equations with only B_1^+

During an RF pulse, the rotating frame Bloch equations become:

$$\frac{d}{dt}\begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} = \begin{pmatrix} -\frac{1}{T_2} & \gamma\Delta B_0 & -\gamma B_{1y}^+ \\ -\gamma\Delta B_0 & -\frac{1}{T_2} & +\gamma B_{1x}^+ \\ \gamma B_{1y}^+ & -\gamma B_{1x}^+ & -\frac{1}{T_1} \end{pmatrix} \begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{10}$$

The red terms are the effects of the RF field. Note that the RF field causes precession around the transverse axis defined by B_1^+, which can be thought of as an effective magnetic field in the rotating frame. This is the basis for how RF pulses manipulate the magnetization vector to create the desired contrast in MRI images.

Note also that because these are complex phasors:

$$B_{1x}^+ = \mathrm{Re}(B_1^+) \text{ is not generally the same as } B_{\mathrm{RFx}} \tag{11}$$

$$B_{1y}^+ = \mathrm{Im}(B_1^+) \text{ is not generally the same as } B_{\mathrm{RFy}} \tag{12}$$

3.5 Bloch equations with gradients

To form an image, we apply magnetic field gradients. These gradients cause the Larmor frequency to vary across space, which allows us to encode spatial information into the MRI signal. The effect of a gradient field $\mathbf{G}(t) = (G_x(t), G_y(t), G_z(t))$ where $G_x = \partial B_{0\mathbf{z}}/\partial x$ etc is to add a position-dependence to the Bloch equations:

$$\frac{d}{dt}\begin{pmatrix} M_x(\mathbf{r}) \\ M_y(\mathbf{r}) \\ M_z(\mathbf{r}) \end{pmatrix} = \begin{pmatrix} -\frac{1}{T_2} & \gamma\Delta B_0 + \gamma\mathbf{G}(t)\cdot\mathbf{r} & -\gamma B_{1y}^+ \\ -\gamma\Delta B_0 - \gamma\mathbf{G}(t)\cdot\mathbf{r} & -\frac{1}{T_2} & +\gamma B_{1x}^+ \\ \gamma B_{1y}^+ & -\gamma B_{1x}^+ & -\frac{1}{T_1} \end{pmatrix} \begin{pmatrix} M_x(\mathbf{r}) \\ M_y(\mathbf{r}) \\ M_z(\mathbf{r}) \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{13}$$

3.6 Summary

The rotating frame Bloch equations capture:

- Rotation around the z-axis due to the frequency offset $\gamma\Delta B_0$
- Plus rotation **around the z-axis** due to gradients $\gamma\mathbf{G}(t)\cdot\mathbf{r}$.
- Rotation around a transverse axis defined by B_1^+ due to RF pulses.
- Longitudinal relaxation of M_z at rate $1/T_1$ towards M_0.
- Transverse relaxation of M_x and M_y at rate $1/T_2$ towards zero.

4 Solving the Bloch equations

4.1 Not by Runge-Kutta

The Bloch equations are coupled ordinary differential equations (ODEs). It is tempting to apply standard ODE solvers such as Runge-Kutta, e.g. `scipy.integrate.solve_ivp`

using `method=RK45` (DEMO1). But this is numerically inefficient because the magnetization changes on a microsecond timescale during RF pulses requiring very small time steps to solve accurately, but this is highly inefficient during periods of free precession where the magnetization changes very slowly and predictably.

4.2 Exponential time step method

Eq 13 can be solved more efficiently by separating it into:

$$\frac{\mathrm{d}\mathbf{M}}{\mathrm{d}t} = \mathsf{A}\mathbf{M} + \mathbf{b} = \underbrace{\mathsf{A}_{\mathrm{rot}}\mathbf{M}}_{\substack{\text{RF, } B_0 \text{ offset,}\\ \text{and gradient terms}}} + \underbrace{\mathsf{A}_{\mathrm{rel}}\mathbf{M} + \mathbf{b}}_{\text{relaxation terms}} \tag{14}$$

where

$$\mathsf{A}_{\mathrm{rot}} = \begin{pmatrix} 0 & \gamma\Delta B_0 + \gamma\mathbf{G}(t)\cdot\mathbf{r} & -\gamma B_{1y}^{+} \\ -\gamma\Delta B_0 - \gamma\mathbf{G}(t)\cdot\mathbf{r} & 0 & +\gamma B_{1x}^{+} \\ \gamma B_{1y}^{+} & -\gamma B_{1x}^{+} & 0 \end{pmatrix} \tag{15}$$

is a skew-symmetric matrix that *generates* rotations when exponentiated (see below) and

$$\mathsf{A}_{\mathrm{rel}} = \begin{pmatrix} -\frac{1}{T_2} & 0 & 0 \\ 0 & -\frac{1}{T_2} & 0 \\ 0 & 0 & -\frac{1}{T_1} \end{pmatrix} \qquad \text{and} \qquad \mathbf{b} = \begin{pmatrix} 0 \\ 0 \\ M_0/T_1 \end{pmatrix} \tag{16}$$

are pure relaxation terms.

The rotation term can be solved analytically. First, consider the long time $t \to \infty$ limit

$$\mathsf{A}\mathbf{M}_\infty + \mathbf{b} = 0 \qquad \text{i.e. typically } \mathbf{M}_\infty = (0, 0, M_0)^T. \tag{17}$$

Defining the deviation from the long-time limit as $\tilde{\mathbf{M}} = \mathbf{M} - \mathbf{M}_\infty$ simplifies Eq 14 to

$$\frac{\mathrm{d}\tilde{\mathbf{M}}}{\mathrm{d}t} = \mathsf{A}\tilde{\mathbf{M}} \tag{18}$$

Assuming RF and gradients are (approximately) constant over an interval $t \to t + \tau$, then

$$\tilde{\mathbf{M}}(t+\tau) = e^{\mathsf{A}(t)\tau}\tilde{\mathbf{M}}(t) \tag{19}$$

and hence

$$\mathbf{M}(t+\tau) = e^{\mathsf{A}(t)\tau}(\mathbf{M}(t) - \mathbf{M}_\infty) + \mathbf{M}_\infty. \tag{20}$$

The matrix exponential $e^{\mathsf{A}(t)\tau}$ can be computed efficiently by factorising it

$$\exp(\mathsf{A}(t)\tau) = \exp\left((\mathsf{A}_{\mathrm{rot}}(t) + \mathsf{A}_{\mathrm{rel}})\tau\right) \approx \exp(\mathsf{A}_{\mathrm{rel}}\tau)\exp(\mathsf{A}_{\mathrm{rot}}(t)\tau). \tag{21}$$

This approximation is exact for constant RF and gradients during τ, and is generally a good approximation even for non-constant fields because relaxation rates are much slower than the

RF- and gradient-induced rotations so $[\mathsf{A}_{\text{rot}}, \mathsf{A}_{\text{rel}}] \approx 0$.

The relaxation term A_{rel} is a diagonal matrix and T_1 and T_2 are constants so

$$\mathsf{E}(\tau) = \exp(\mathsf{A}_{\text{rel}}\tau) = \begin{pmatrix} e^{-\tau/T_2} & 0 & 0 \\ 0 & e^{-\tau/T_2} & 0 \\ 0 & 0 & e^{-\tau/T_1} \end{pmatrix}. \tag{22}$$

The rotation term A_{rot} is a skew-symmetric matrix so its exponential can be computed using the Rodrigues' rotation formula[2]

$$\mathsf{R}(t,\tau) = \exp(\mathsf{A}_{\text{rot}}(t)\tau) = \mathsf{I} + \frac{\sin\theta}{\theta}\mathsf{A}_{\text{rot}}(t)\tau + \frac{1-\cos\theta}{\theta^2}\mathsf{A}_{\text{rot}}^2(t)\tau^2 \tag{23}$$

where $\theta = \tau\sqrt{-\frac{1}{2}\text{Tr}(\mathsf{A}_{\text{rot}}^2)}$ is the angle of rotation.

Putting this all together, Eq 20 simplifies to:

$$\mathbf{M}(t+\Delta t) = R(t,\tau)\mathsf{E}(\tau)(\mathbf{M}(t) - \mathbf{M}_\infty) + \mathbf{M}_\infty. \tag{24}$$

To make simulations, we *discretise* time into small intervals. During each interval, we assume that the RF and gradients are constant so that the above formula applies. We can then iterate this formula to simulate the magnetization dynamics over time. This is known as the **exponential time step method**.

The final time-step update formula for $\mathbf{M}_n \equiv \mathbf{M}(t_n)$ is:

$$\mathbf{M}_{n+1} = \mathsf{E}(\tau_n)\,\mathsf{R}(t_n)\mathbf{M}_n + \mathbf{c}_n \qquad \text{with } \tau_n = t_{n+1} - t_n \tag{25}$$

where $\mathbf{c}_n = (\mathsf{I} - \mathsf{E}(\tau_n))\,\mathbf{M}_\infty$.[3]

The terms E represent the effects of relaxation, the terms R represent the effects of RF and gradients, and the term $\mathbf{c}_n$ represents the effect of relaxation towards the long-time limit $\mathbf{M}_\infty$. This method is much more efficient than Runge-Kutta because it is exact for arbitrarily long time steps during periods of free precession, while still accurately capturing the rapid changes during RF pulses.

This method is implemented in the popular Matlab `bloch_CTR_Hz` function[4] and the `bloch-simulator-python` translation.[5]

4.3 Advanced methods

The details are well beyond the scope of this course, but there are more advanced methods for solving the Bloch equations that can be even more efficient and accurate. These include:

- Evaluating the A_{rot} matrices using the Cayley-Hamilton theorem, which allows us to

[2] https://mathworld.wolfram.com/RodriguesRotationFormula.html

[3] Strictly, this assumes negligible relaxation occurs during RF pulses. This is true for all normal MRI sequences since RF pulses last milliseleconds while relaxation times are much longer. Gradients and off resonance do not affect this approximation since they only rotate about the z-axis.

[4] https://pmc.ncbi.nlm.nih.gov/articles/PMC5076794/ which fixes some unit issues and axis conventions from the original https://web.archive.org/web/20180814160141/http://www-mrsrl.stanford.edu/~brian/blochsim/bloch.c

[5] https://github.com/namalkanti/bloch-simulator-python

compute the matrix exponential without explicitly computing the Rodrigues' rotation formula.

- Evaluating the A_{rot} matrices using Quaternions, which can be more numerically stable and efficient for large rotations, and are implemented directly in hardware on GPUs.
- The Extended Phase Graph (EPG) formalism which simulates an entire 3D object using a discrete Fourier transform.
- The Phase Distribution Graph (PDG) formalism which simulates an entire 3D object using a discrete Fourier transform but also captures the effects of diffusion and other non-idealities that are not captured by EPG.

The open source `MRzero` software that we will use applies all these tricks internally.

5 Simulating the pulse-acquire sequence

The pulse-acquire sequence is a simple MRI sequence that consists of an RF pulse followed by a period of free precession during which the signal is acquired. Fig. 1 shows the output of an `MRzero` simulation of a FID sequence. A single RF pulse is applied, which causes a decaying oscillating signal.[6]

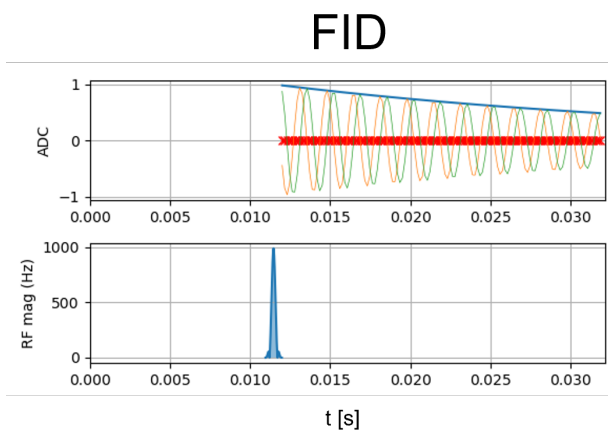

Figure 1: The pulse-acquire sequence. The RF pulse is applied at time $t = 0$ and the signal is acquired during the subsequent free precession period. The signal decays due to T2 relaxation, and the frequency of the signal is determined by the Larmor frequency and any frequency offsets.

If apply an inverse Fourier transform to this signal, we recover a single Lorentzian-shaped peak in the spectrum, as shown in Fig 2.

[6]See https://github.com/ctr/medical-physics-demos-2026-03/ for code.

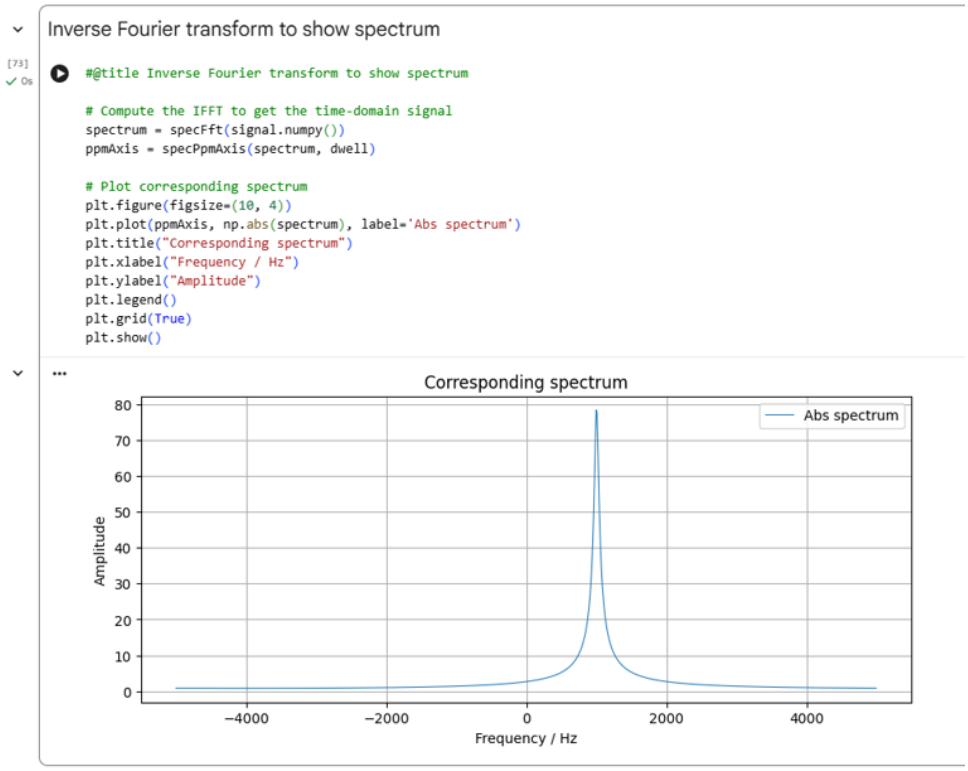

Figure 2: Python reconstruction of the spectrum from the FID signal.

6 Reconstruction of a 2D Gradient Echo Scan

In the lecture, I show a simulation of a 2D imaging sequence: gradient-recalled echo (GRE). After the simulation (or scan) has completed we have a succession of acquired k-space data points. Each k-space point corresponds to a particular combination of phase-encoding and frequency-encoding gradients that were applied during the scan.

We can assemble this data together as elements of a complex k-space matrix $K_{m,n}$, where m indexes the phase-encoding steps and n indexes the sample number within each readout (frequency-encoding). The size of this matrix is determined by the number of phase-encoding and frequency-encoding steps that were acquired during the scan. In our exxample, $K_{m,n}$ was a 64×64 complex matrix.

We choose to follow python conventions and index from zero, so $m = 0, \ldots, M-1$ and $n = 0, \ldots, N-1$ where M and N are the number of phase-encoding and frequency-encoding steps, respectively.

The reconstructed image $I_{x,y}$ is obtained via the 2D discrete inverse Fourier transform (2D-IDFT):[1, 5]

$$I_{x,y} = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} K_{m,n} e^{i2\pi\left(\frac{mx}{M} + \frac{ny}{N}\right)}, \quad x = 0, \ldots, M-1, \quad y = 0, \ldots, N-1. \tag{26}$$

In practice, this reconstruction is performed efficiently using the 2D inverse fast Fourier transform (iFFT). In Python, using `numpy`, this can be implemented as:

```
import numpy as np
```

```
# K is the acquired k-space data of shape (M, N)
# fftshift moves the zero-frequency component
# to the corner of K and then to unpack the image
I = np.fft.fftshift(np.fft.ifft2(np.fft.fftshift(K)))

# Take magnitude for the final image
I_mag = np.abs(I)
```

The `ifft2` function computes the 2D inverse FFT efficiently, avoiding the $\mathcal{O}(M^2N^2)$ complexity of the direct IDFT.

References

1. Haacke, E. M. *Magnetic resonance imaging : physical principles and sequence design* xxvii, 914 p. ISBN: 0471351288 (Wiley-Liss, New York ; Chichester, 1999).
2. Levitt, M. H. *Spin dynamics : basics of nuclear magnetic resonance* 2nd, xxv, 714 p., 7 p. of plates. ISBN: 9780470511183 (John Wiley & Sons, Chichester, England ; Hoboken, NJ, 2008).
3. Slichter, C. *Principles of Magnetic Resonance* ISBN: 978-3-642-08069-2. doi:10.1007/978-3-662-09441-9 (1990).
4. Hoult, D. I. The principle of reciprocity in signal strength calculations - A mathematical guide. *Concepts in Magnetic Resonance* **12.** 325LL Times Cited:189 Cited References Count:11, 173–187. ISSN: 1043-7347. doi:Doi10.1002/1099-0534(2000)12:4<173::Aid-Cmr1>3.0.Co;2-Q (2000).
5. Bracewell, R. N. *The Fourier transform and its applications* 3rd ed. Ronald N. Bracewell. ISBN: 0073039381 0071160434 CIP entry (WCB/McGraw Hill, Boston, 2000).